

Atos Origin™

CONSULTING > SOLUTIONS > OUTSOURCING

JavaScript côté serveur et en mode déconnecté

Solutions Linux 2009

Raphaël SEMETEYS
raphael.semeteys@atososrigin.com

31/03/2009

Atos, Atos et le poisson, Atos Origin et le poisson, Atos Consulting ainsi que le poisson seul sont des marques déposées d'Atos Origin SA.
© 2009 Atos Origin. Document exclusivement réservé à usage commercial. Tout ou partie de ce document ne peut être copié, modifié, diffusé ou annoté sans accord écrit d'Atos Origin ou du client.

ADVANCE YOUR BUSINESS »

Sommaire

JavaScript coté serveur

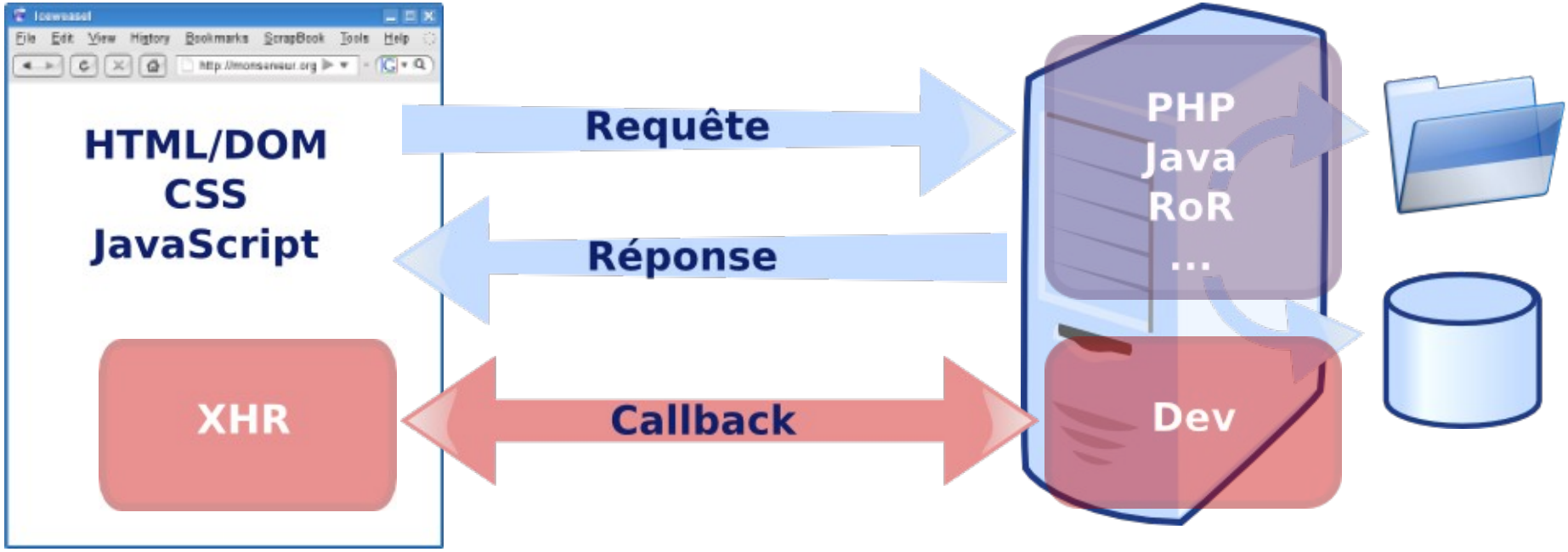


Gestion du mode déconnecté



ADVANCE YOUR BUSINESS »

Architectures Web traditionnelles



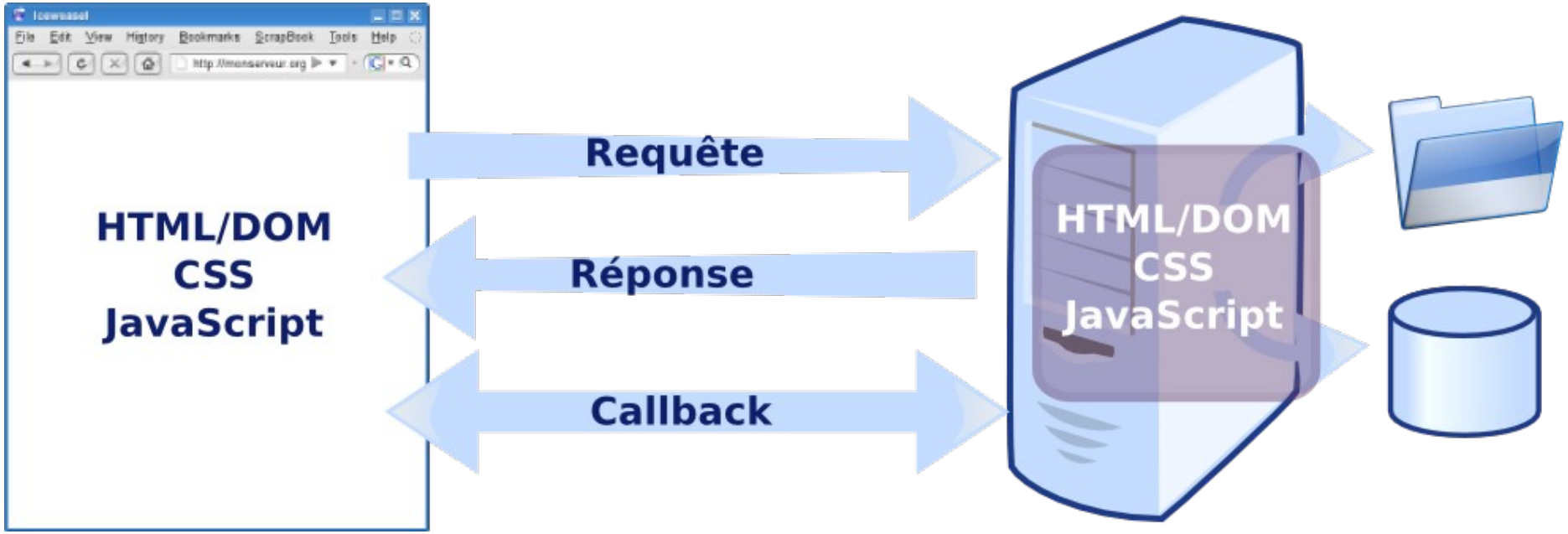
Rupture technologique entre le client et le serveur

Développements spécifiques autour de XHR et coté serveur

Pas de mode déconnecté

ADVANCE YOUR BUSINESS »

Serveur Ajax

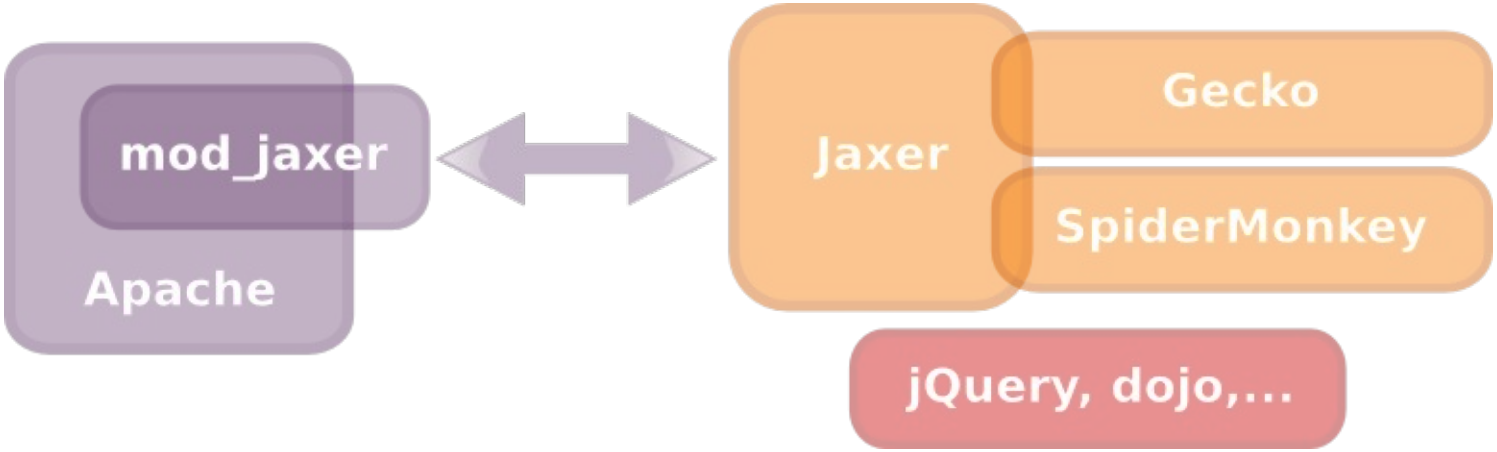


Aptana Jaxer (http://www.aptana.com/jaxer)

Code AJAX exécuté avant que la page ne soit retournée au navigateur Web

Proxy functions pour appeler directement des fonctions serveur à partir du client

Composition



GPL

API Jaxer :

- Jaxer.DB (SQLite, MySQL)
- Jaxer.SMTP, Jaxer.NetworkUtils, Jaxer.Socket
- Jaxer.File, Jaxer.Dir et Jaxer.DirUtils
- Jaxer.System, Jaxer.Process, Jaxer.Thread
- Jaxer.Request, Jaxer.Response

Chargement de farmeworks externes

ADVANCE YOUR BUSINESS »

Utilisation basique de Jaxer (1/3)

Exemple : gestion basique de liste en base de données

Code coté serveur :

```
<script runat="server">
    Jaxer.DB.execute("CREATE TABLE IF NOT EXISTS maliste (id INTEGER PRIMARY KEY AUTO_INCREMENT, name
TEXT)");

    function save(name) {
       Jaxer.DB.execute("INSERT INTO maliste (name) VALUES (?)", name);
    }
    save.proxy = true;

    function suppr(id) {
       Jaxer.DB.execute("DELETE FROM maliste WHERE id=(?)", id);
    }
    suppr.proxy = true;

    function supprTous() { Jaxer.DB.execute("DELETE FROM maliste"); }
    supprTous.proxy = true;

    function getList() {
       return Jaxer.DB.execute("SELECT * FROM maliste");
    }
    getList.proxy = true;
</script>
```

`runat="server"`

Code exécuté par Jaxer
Option : `runat="both"`

`Jaxer.*()`

Appels d'API Jaxer
Ici : base SQLite embarquée

`getList.proxy = true;`

Déclaration d'une proxy function

ADVANCE YOUR BUSINESS »

Utilisation basique de Jaxer (2/3)

Code coté client :

```
<script>
    function $(id) { return document.getElementById(id); }

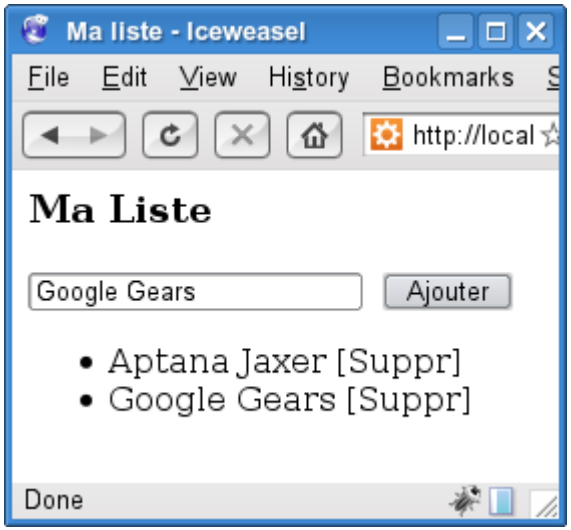
    function showList() { getList.async(showResult); }

    function showResult(resultset) {
       var results = new Array;
       resultset.rowsAsArrays.forEach(function(row) {
          results.push(new Array(row[0], row[1]));
       });
       var html;
       if (results.length) {
          html = "<ul>";
          for (i=0; i<results.length; i++) {
             html += "<li>" + results[i][1] +
                        " <span onclick='suppr(" +
                     results[i][0] + "); showList();'>[Suppr]</span>" +
                        "</li>";
          }
          html += "</ul>";
       } else html = "<br/>Liste vide...";
       $("liste").innerHTML = html;
    }
    showList();
 </script>
```

`getList.async(showResult);`

Appel asynchrone de la Proxy function

Démonstration



Démonstration
si KO : demo.sh

ADVANCE YOUR BUSINESS »

Utilisation basique de Jaxer (3/3)

Page envoyée au navigateur (view source) :

```
<html><head>
  <script type="text/javascript" src="/jaxer/framework/clientFramework_compressed.js?version=1.0.3.4547"></script>
  <script type="text/javascript">
    Jaxer.Callback.pageSignature = 351955322;
    Jaxer.Callback.pageName = 'localhost:8081/solinux/index.html';
    Jaxer.Callback.callingPage = 'http://localhost:8081/solinux/index.html';
    Jaxer.CALLBACK_URI = '/jaxer-callback';
    Jaxer.ALERT_CALLBACK_ERRORS = false;
</script>
<title>Ma liste</title>
</head><body>
  <h3>Ma Liste</h3>
  <input id="name" type="text"/>
  <input value="Ajouter" onclick="save($('name').value); showList();" type="button"/>
  <div id="liste"/>
  <script>
    eval(Jaxer.Callback.createProxies(['save', 'suppr', 'deleteAll', 'getList']));
  </script>
  <script>
    function $(id) { return document.getElementById(id); }
    function showList() { getList.async(showResult); }
    [...]
  </script>
</body></html>
```

`clientFramework_compressed.js`

Partie cliente de Jaxer

`Jaxer.Callback.*`

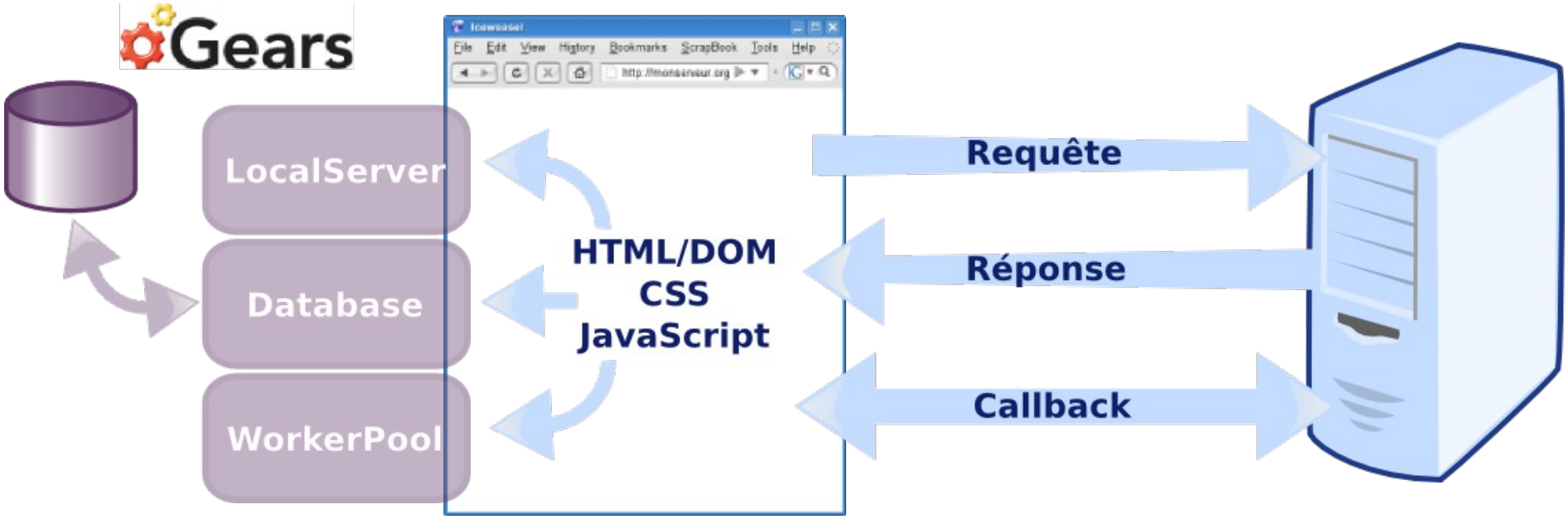
Données pour le callback

`eval(Jaxer.Callback.createProxies(..))`

Création des proxies

ADVANCE YOUR BUSINESS »

Google Gears



Mode déconnecté : stockage local de pages statiques et de données (SQLite)

Améliorer les navigateurs en attendant HTML 5

Utilisation basique de Gears (1/2)

Modification du code client

Initialisation de Gears : `<script type="text/javascript" src="gears_init.js"></script>`

Initialisation de l'application :

```
var local = false;
var db = google.gears.factory.create("beta.database");
db.open("maliste");
db.execute(
  "CREATE TABLE IF NOT EXISTS maliste (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT)"
);
var localServer = google.gears.factory.create('beta.localserver');
var store = localServer.createStore("maliste");
store.enabled = false;
store.capture(["gears.html"], null);
```

Ajout de traitements locaux :

```
function dispatchSave(name) { (local)?saveLocal(name):save(name); }

function saveLocal(name) {
  db.execute("INSERT INTO maliste (name) VALUES (\"" + name + "\")"
  );
}
```


ADVANCE YOUR BUSINESS »

Utilisation basique de Gears (2/2)

Modification du code client (suite)

Ajout d'un bouton : `<input type="button" value="Mode" onclick="doSync();"/><span id="mode"> : online</span>`

Gestion de la synchronisation :

```
function doSync() {
  if (local) {
    suppTous();
    var resultset = getListLocal();
    while (resultset.isValidRow()) {
      save(resultset.field(1));
      resultset.next();
    }
    local = false;
    $("mode").textContent = " : online";
    store.enabled = false;
  } else {
    db.execute("DELETE FROM maliste");
    var resultset = getList();
    resultset.rowsAsArrays.forEach(
      function(row) {
        saveLocal(row[1]);
      }
    );
    local = true;
    $("mode").textContent = " : offline";
    store.enabled = true;
  }
}
```

`save(...)`

Déconnexion
Appels distants (proxy function)

`saveLocal(...)`

Reconnexion
Appels locaux (Gears)

Démonstration
si KO : demo.sh

ADVANCE YOUR BUSINESS »

Conclusion

Il est possible d'enrichir les applications Web

- En respectant les **standards**
  - HTML/DOM/CSS
  - ECMAScript
- En utilisant des produits **libres**
  - Coté serveur : Aptana Jaxer
  - Coté client : Google Gears
- En unifiant les langages et frameworks de programmation

Points d'**attention**

- Les callbacks ou le mode déconnecté remettent en cause la logique applicative
- C'est un peu « cuting edge », mais est-ce plus risqué que de tuer HTML 5 ?